

Лабораторная работа № 8 по курсу дискретного анализа: Графы

Выполнил студент группы М80-308Б-22 МАИ *Цирулев Николай*.

Условие

Вариант №8: Поиск максимального паросочетания алгоритмом Куна

Ограничение времени	10 секунд
Ограничение памяти	1 Gb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Задан неориентированный двудольный граф, состоящий из n вершин и m ребер. Вершины пронумерованы целыми числами от 1 до n . Необходимо найти максимальное паросочетание в графе алгоритмом Куна. Для обеспечения однозначности ответа списки смежности графа следует предварительно отсортировать. Граф не содержит петель и кратных ребер.

Формат ввода

В первой строке заданы $1 \leq n \leq 110000$ и $1 \leq m \leq 40000$. В следующих m строках записаны ребра. Каждая строка содержит пару чисел - номера вершин, соединенных ребром.

Формат вывода

В первой строке следует вывести число ребер в найденном паросочетании. В следующих строках нужно вывести сами ребра, по одному в строке. Каждое ребро представляется парой чисел - номерами соответствующих вершин. Строки должны быть отсортированы по минимальному номеру вершины на ребре. Пары чисел в одной строке также должны быть отсортированы.

Метод решения

Задан двудольный граф $G\langle V, E \rangle$, для которого требуется найти наибольшее паросочетание. Алгоритм заключается в следующем: начиная с пустого паросочетания, пока удаётся найти увеличивающую цепь, выполняем чередование паросочетания вдоль этой цепи. Процесс продолжается до тех пор, пока не удастся найти увеличивающую цепь.

В массиве mt хранятся паросочетания, где $mt[v]$ — вершина, с которой связано v , или -1 , если паросочетания нет. Массив u отслеживает посещённость вершин в процессе поиска увеличивающих цепей.

Функция `dfs` находит увеличивающую цепь, чередует паросочетания вдоль этой цепи и возвращает `true`, если такая цепь найдена. Внутри `dfs` проверяется, может ли

ребро привести к ненасыщенной вершине или если насыщенная вершина имеет увеличивающую цепь через уже занятое паросочетание.

В `main` вершины $v \in V$, для каждой выполняет `dfs`, обнуляя массив u . Массив mt после завершения программы содержит искомое паросочетание.

Описание программы

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

bool dfs(int v, vector<bool>& u, vector<int>& mt, const vector<vector<int>>& g) {
    if (u[v]) {
        return false;
    }
    u[v] = true;
    for (int to : g[v]) {
        if (mt[to] == -1 || dfs(mt[to], u, mt, g)) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

int main() {
    int n, m;
    cin >> n >> m;

    vector<vector<int>> g(n);
    vector<int> mt(n, -1);
    vector<bool> u(n);
    int a, b;
    while (cin >> a >> b) {
        g[a - 1].push_back(b - 1);
        g[b - 1].push_back(a - 1);
    }
    for (int i = 0; i < n; ++i) {
        sort(g[i].begin(), g[i].end());
    }
}
```

```

    for (int v = 0; v < n; ++v) {
        fill(u.begin(), u.end(), false);
        dfs(v, u, mt, g);
    }
    vector<pair<int, int>> res;
    for (int i = 0; i < n; ++i) {
        if (mt[i] != -1 && mt[i] < i) {
            res.emplace_back(mt[i] + 1, i + 1);
        }
    }
    sort(res.begin(), res.end());
    cout << res.size() << endl;
    for (const auto& [a, b] : res) {
        cout << a << ' ' << b << endl;
    }

    return 0;
}

```

Дневник отладки

Судя по всему сначала решение не проходило чекер на 6-ом тесте из-за того, что предварительно не были отсортированы списки смежности графа.

Тест производительности

Асимптотика написанного алгоритма - $O(MN)$. Для лучшей наглядности приведём таблицу, в которой время работы алгоритма сопоставляется с вводимыми данными. Для удобства расчетов сделаем $M = N^2$.

N	Время работы алгоритма, мс
10	106
100	32290
1000	27484511

Как мы видим по таблице, реальное время работы приближается к теоретической временной сложности алгоритма, большее приближение можно увидеть при больших данных. Ниже приведена программа `benchmark.cpp`, использовавшаяся для определения времени работы алгоритма:

```

#include <iostream>
#include <vector>
#include <algorithm>

```

```

#include <random>
#include <chrono>

using namespace std;

bool dfs(int v, vector<bool>& u, vector<int>& mt, const vector<vector<int>>& g) {
    if (u[v]) {
        return false;
    }
    u[v] = true;
    for (int to : g[v]) {
        if (mt[to] == -1 || dfs(mt[to], u, mt, g)) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

```

```

int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    for (int j = 10; j < 1e10; j *= 10) {
        std::uniform_int_distribution int_dist(0, j - 1);
        std::cout << j << "_&_";
        auto start = std::chrono::high_resolution_clock::now();
        int n = j;
        vector<vector<int>> g(n);
        vector<int> mt(n, -1);
        vector<bool> u(n);
        for (int i = 0; i < n*n; i++) {
            int a = int_dist(gen);
            int b = int_dist(gen);
            g[a].push_back(b);
            g[b].push_back(a);
        }
        for (int i = 0; i < n; ++i) {
            sort(g[i].begin(), g[i].end());
        }
        for (int v = 0; v < n; ++v) {

```

```

        fill(u.begin(), u.end(), false);
        dfs(v, u, mt, g);
    }
    vector<pair<int, int>> res;
    for (int i = 0; i < n; ++i) {
        if (mt[i] != -1 && mt[i] < i) {
            res.emplace_back(mt[i] + 1, i + 1);
        }
    }
    sort(res.begin(), res.end());

    auto finish = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::microsecond>
        (finish - start);
    std::cout << duration.count() << "_\\\\" << "\\_";
    std::cout << '\\n';
}
return 0;
}

```

Выводы

В ходе выполнения данной работы была изучена теория графов и на практике реализован алгоритм Куна нахождения наибольшего паросочетания в неориентированном двудольном графе.

Теория графов очень широко применяется для решения реальных задач, таких как, например, поиск кратчайшего пути или же реконструкция РНК.

Полученные знания могут служить основой для дальнейшего изучения теории графов и применения ее методов к реальным задачам.